

Corso: Programmazione di sistema

Quiz: API Programming - Teoria 8 settembre 2026



PORTALE
ESAMI

	Nicola Beccaceci s360641
Stato	Completato
Iniziato	martedì, 8 settembre 2026, 12:30
Terminato	martedì, 8 settembre 2026, 13:14
Tempo impiegato	43 min. 44 secondi
Valutazione	7,6 su un massimo di 9,0 (84,4%)
Riepilogo del tentativo	1 2 3 ● ● ●

Domanda 1

Completo

Punteggio
ottenuto 2,8
su 3,0

Dato il seguente programma:

```
fn main() {  
    let v = vec![1, 2, 3, 4, 5, 6];  
    let it = v.iter()  
        .map(|x| {  
            println!("  map {}", x);  
            x * 2  
        })  
        .filter(|x| x % 3 == 0);  
    println!("iteratore creato");  
  
    let r: Vec<i32> = it.collect();  
    println!("r = {:?}", r);  
  
    let primo = v.iter()  
        .map(|x| x * 10)  
        .find(|x| *x > 25);  
    println!("primo = {:?}", primo);  
  
    let s: i32 = v.iter().take(3).sum();  
    println!("s = {}", s);  
  
    let mut w = vec![String::from("a"), String::from("b")];  
    for x in w.iter_mut() { x.push('!'); }  
    println!("w = {:?}", w);  
  
    let z: Vec<String> = w.into_iter().collect();  
    println!("z = {:?}", z);  
}
```

- A. Si scriva l'output completo, rispettando l'ordine delle righe. (1,25)
- B. Perché la riga «iteratore creato» compare prima di «map 1»? (0,5)
- C. Quante volte viene eseguita la closure passata a map nella riga che calcola primo? Perché? (0,5)
- D. Si spieghi la differenza fra `iter()`, `iter_mut()` e `into_iter()`. Dopo la riga che calcola z, la variabile w è ancora utilizzabile? (0,75)

A. "iteratore creato", "map 1", "map 2", "map 3", "map 4", "map 5", "map 6", "r = [1, 2, 3, 4, 5, 6]", "primo = 30", "sum = 6", "w = [a!, b!]", "z = [a!, b!]"

B. Gli iteratori son lazy, ovvero intraprendono l'operazione richiesta soltanto a fronte di un consumatore. Dal momento che map e filter sono adattatori e non vi è un consumatore finale, verrà stampato "prima iteratore creato" e solo a seguito della collect() verrà stampato "map 1"

C. Viene eseguita tre volte in quando il consumatore find() esce non appena un elemento soddisfa la condizione specificata (in questo caso $x > 25$), la quale è soddisfatta al terzo elemento

D. iter() e iter_mut() prendono a prestito, rispettivamente prestito immutabile e prestito mutabile, gli elementi dell'oggetto iterable su cui sono stati invocati; pertanto, il vettore originale rimane accessibile anche a seguito della serie di operazioni successive al metodo. into_iter() invece consuma l'oggetto su cui viene invocato prendendo il possesso di ciascun elemento; pertanto, al termine delle operazioni l'oggetto originale non è più accessibile (se proviamo ad accedervi riceveremo l'errore di compilazione "use of moved value")

Commento:

1) r = [6, 12], primo = Some(30)

2) OK

3) OK

4) OK

Domanda 2

Completo

Punteggio
ottenuto 2,7
su 3,0

Dato il seguente programma:

```

use std::cell::RefCell;
use std::rc::{Rc, Weak};

struct Nodo {
    valore: i32,
    figli: RefCell<Vec<Rc<Nodo>>>,
    padre: RefCell<Weak<Nodo>>,
}

impl Drop for Nodo {
    fn drop(&mut self) {
        println!("drop nodo {}", self.valore);
    }
}

fn main() {
    let radice = Rc::new(Nodo {
        valore: 1,
        figli: RefCell::new(vec![]),
        padre: RefCell::new(Weak::new()),
    });

    println!("A strong={} weak={}", Rc::strong_count(&radice), Rc::weak_count(&radice));

    {
        let figlio = Rc::new(Nodo {
            valore: 2,
            figli: RefCell::new(vec![]),
            padre: RefCell::new(Weak::new()),
        });

        *figlio.padre.borrow_mut() = Rc::downgrade(&radice);
        radice.figli.borrow_mut().push(Rc::clone(&figlio));

        println!("B strong={} weak={}", Rc::strong_count(&radice), Rc::weak_count(&radice));
        println!("C strong={} weak={}", Rc::strong_count(&figlio), Rc::weak_count(&figlio));
        println!("padre del figlio = {:?}",
            figlio.padre.borrow().upgrade().map(|p| p.valore));
        println!("fine blocco interno");
    }

    println!("D strong={} weak={}", Rc::strong_count(&radice), Rc::weak_count(&radice));
    println!("figli della radice = {}", radice.figli.borrow().len());
    println!("fine main");
}

```

- A. Si scriva l'output completo del programma, comprese le stampe prodotte da `drop`. (1)
- B. Perché i campi figli e padre sono racchiusi in un `RefCell`? Quale controllo si sposta dal tempo di compilazione al tempo di esecuzione e che cosa accade se viene violato? (1)
- C. Che cosa cambierebbe se il campo padre fosse dichiarato `RefCell<Option<Rc<Nodo>>>` e si memorizzasse `Some(Rc::clone(&radice))`? Si giustifichi la risposta. (1)

A. "A strong=1, weak=0", "B strong=1, weak=1", "C strong=2, weak=0", "padre del figlio = 1", "fine blocco interno", "drop nodo 2", "D strong=1, weak=0", "figli della radice = 0", "fine main", "Drop nodo 1"

B. I campi figi e padre sono racchiusi in una refcell al fine di rilassare i controlli del borrow checker, passando da controlli/errori a compile time a controlli/errori a run time. Questo è necessario programmi logicamente corretti potrebbero scontrarsi con il 'conservatorismo' del borrow checker. Il controllo eseguito a run time è quello dei prestiti: possono esistere più prestiti immutabili oppure al più un prestito mutabile per uno stesso oggetto. Pertanto, il risultato finale sarà quello di spostare i controlli effettuati dal borrow checker da compile time a run time. Qualora uno di questi controlli fallisse, genererebbe un panic!() a runtime (invece che un errore di compilazione).

C. Questo, anche se permesso dal borrow checker, causerebbe un memory leak in quanto padre e figlio avrebbero entrambi strong_count = 2 nell'istante precedente al drop. Pertanto, a seguito del decremento dello strong_counter causato dall'invocazione del metodo drop(), entrambi i valori continueranno ad essere maggiori di 0 (pari a 1) e pertanto non verrà eseguita alcuna deallocazione

Commento:

1) Output corretto:

```
A strong=1 weak=0
B strong=1 weak=1
C strong=2 weak=0
padre del figlio = Some(1)
fine blocco interno
D strong=1 weak=1
figli della radice = 1
fine main
drop nodo 1
drop nodo 2
```

2) OK

3) OK

Domanda 3

Completo

Punteggio
ottenuto 2,1
su 3,0

Dato il seguente programma:

```

use std::sync::mpsc::sync_channel;
use std::thread;
use std::time::Duration;

fn main() {
    let (tx, rx) = sync_channel::<u32>(2);

    for i in 1..=5 {
        println!("invio {}", i);
        tx.send(i).unwrap();
        println!("inviato {}", i);
    }

    let consumatore = thread::spawn(move || {
        let mut somma = 0;
        for v in rx {
            thread::sleep(Duration::from_millis(10));
            somma += v;
        }
        somma
    });

    let somma = consumatore.join().unwrap();
    println!("somma = {}", somma);
}

```

- A. Cosa si osserva sulla console quando si lancia il programma? (1)
- B. Che differenza c'è fra `mpsc::channel()` e `mpsc::sync_channel(k)`? Che cosa si intende per backpressure e a che cosa serve? (0,75)
- C. Si corregga il programma affinché stampi la somma dei valori inviati. (0,75)
- D. Nella versione corretta, perché è necessario `drop(tx)`? Che cosa restituisce `recv()` quando tutti i sender sono stati distrutti, e che cosa restituisce `send()` se il receiver non esiste più? (0,5)

A Il programma non termina: si blocca. Questo è dovuto al fatto che il trasmettitore non viene distrutto prima che si arrivi all'istruzione `consumatore.join().unwrap()`. Mantenendo in vita il trasmettitore, manteniamo in vita anche il canale e di conseguenza il ricevitore, il quale, a sua volta, non uscirà mai dal `for`. Dal momento che l'istruzione `consumatore.join().unwrap()` attende la terminazione del thread spawnato (e questo resta bloccato nel ciclo `for`), il programma non terminerà mai.

B. I canali `mpsc::channel()` sono canali unbounded, ovvero i trasmettitori possono inviare quanti oggetti vogliono e a che frequenza vogliono, indipendentemente dal ricevitore. Al fine di evitare di saturare la memoria a causa di un gap tra la frequenza di invio e quella di ricezione è possibile definire dei canali bounded (`sync_channel`), i quali implementano il meccanismo della backpressure: bloccano i trasmettitori se il numero di elementi inviati ma non ancora ricevuti è uguale a `k`. Pertanto, nel primo caso la `send()` è non bloccante, mentre nel secondo (bounded channels) può risultare bloccante.

C. È sufficiente inserire l'istruzione `drop(tx)` al termine del primo `for` (`for i in 1..=5`) o comunque prima dell'istruzione `consumatore.join().unwrap`, in quanto questa attende la terminazione del thread spawnato.

D. È necessario inserire `drop(tx)` al fine di segnalare al ricevitore che tutti i trasmettitori sono stati droppati, pertanto, il canale è chiuso. Sia la `send()` che la `recv()` restituiscono un `Result<>`. Nel caso della `send()` questa restituirà `Ok(())` quando il canale è aperto e il messaggio è stato inviato, `Err` se il ricevitore si è disconnesso. Nel caso della `recv()`, questa restituirà `Ok(u32)` se uno dei trasmettitori ha inviato un dato, mentre `Err` se tutti i trasmettitori sono stati droppati.

Commento:

1) Il programma rimane bloccato all'infinito ma molto prima di arrivare alla fine del main, in quanto si cercano di inviare 5 elementi su un canale di capacità 2 che non ha lettori attivi

2) OK

3) No, non è sufficiente, va anche spostata la creazione del thread consumatore prima di iniziare ad inviare messaggi

4) OK